



Apellido y Nombres	Legajo	# de Hojas	Profesor

Normas generales:

Numere las hojas entregadas. **Lea detenidamente cada pregunta y consulte las dudas de interpretación que pudieran surgir.** Complete en la primera hoja la cantidad total de hojas entregadas. Debe identificarse cada hoja con: nombres, apellido, legajo, y nombre del profesor del curso. **Por favor entregar esta hoja y las restantes del tema junto al examen.**

Sección Teórica

1. Indique si las siguientes afirmaciones son verdaderas o falsas:

Una instancia de un programa creado utilizando la función <code>pthread_create</code> tiene el mismo process ID (PID) que el proceso que lo originó.	V	F
Dentro de un proceso puede haber diferentes hilos de ejecución (various threads) los cuales comparten el mismo espacio de memoria.	V	F
Evitamos que un proceso entre en estado Z liberando correctamente los recursos desde el proceso padre o enviando una señal SIGKILL al proceso que finalizó.	V	F

2. Dibujar un diagrama de bloques con la interacción entre un proceso cliente y un servidor y describir su funcionamiento.
3. ¿Cuáles son las diferencias (en términos del mapa de memoria) entre estructuras, uniones y campos de bits? Ejemplifique.

Sección Práctica

- 1a. Implemente una función que permita agregar la información de pacientes que llegan a atenderse a un consultorio médico en una cola de atención. La cola deberá utilizar estructuras autorreferenciadas para almacenar dicha información.

Utilice las siguientes estructuras para almacenar la información:

```
typedef struct Paciente
{
    char nombre[30];
    int edad;
    char obra_social[30];
} Paciente;

typedef struct Nodo
{
    Paciente paciente;
    struct Nodo* siguiente;
} Nodo;
```

Utilice el siguiente prototipo:



Apellido y Nombres	Legajo	# de Hojas	Profesor

```
int encolar(Nodo** cola_pacientes, Paciente paciente);
```

Parámetros:

- cola_pacientes: Cola de pacientes almacenada utilizando estructuras autorreferenciadas.
- paciente: Datos del paciente a agregar en la cola.

Valor de retorno:

- ERROR (-1) en caso de error.
- EXITO (0) si se pudo encolar el paciente satisfactoriamente.

1b. Implemente una función que permita agregar la información de pacientes que llegan a atenderse a un consultorio médico en un arreglo dinámico de atención.

Utilice la siguiente estructura para almacenar la información:

```
typedef struct Paciente
{
    char nombre[30];
    int edad;
    char obra_social[30];
} Paciente;
```

Utilice el siguiente prototipo:

```
int agregar(Paciente** pacientes, int* cantidad_pacientes, Paciente paciente);
```

Parámetros:

- pacientes: Pacientes almacenados utilizando un arreglo dinámico de estructuras.
- cantidad_pacientes: Cantidad de pacientes en el arreglo dinámico.
- paciente: Datos del paciente a agregar en la cola.

Valor de retorno:

- ERROR (-1) en caso de error.
- EXITO (0) si se pudo agregar el paciente satisfactoriamente.

2. Implemente una función que reciba la cola de pacientes o el arreglo dinámico de pacientes y un listado de obras sociales y guarde en un archivo de texto los datos correspondientes a los pacientes cuya obra social se encuentre presente en la lista. Se deberán utilizar las mismas estructuras definidas en el ejercicio 1a o 1b según corresponda. Tanto la cola o el arreglo dinámico no deberán sufrir modificación alguna en esta función. El archivo de texto deberá ser de formato CSV, es decir, deberá poseer una fila por cada paciente y la información de cada paciente deberá estar separada por comas (',').

Utilice el siguiente prototipo si realizó previamente el **ejercicio 1a**:



Apellido y Nombres	Legajo	# de Hojas	Profesor

```
int filtrar(Nodo* cola_pacientes, char** obras_sociales, int
cantidad_obras_sociales, char* nombre_archivo);
```

Parámetros:

- cola_pacientes: Cola de pacientes almacenada utilizando estructuras autorreferenciadas.
- obras_sociales: Lista de obras sociales almacenada como un arreglo de strings.
- cantidad_obras_sociales: Cantidad de obras sociales en la lista.
- nombre_archivo: Nombre del archivo a generar.

Valor de retorno:

- ERROR (-1) en caso de error.
- La cantidad de pacientes guardados en el archivo si el mismo se pudo generar satisfactoriamente.

Utilice el siguiente prototipo si realizó previamente el **ejercicio 1b**:

```
int filtrar(Pacientes* pacientes, int cantidad_pacientes, char** obras_sociales,
int cantidad_obras_sociales, char* nombre_archivo);
```

Parámetros:

- pacientes: Arreglo de pacientes.
- cantidad_pacientes: Cantidad de pacientes en el arreglo.
- obras_sociales: Lista de obras sociales almacenada como un arreglo de strings.
- cantidad_obras_sociales: Cantidad de obras sociales en la lista.
- nombre_archivo: Nombre del archivo a generar.

Valor de retorno:

- ERROR (-1) en caso de error.
- La cantidad de pacientes guardados en el archivo si el mismo se pudo generar satisfactoriamente.

3. Implemente un programa que reciba por argumentos de línea de comandos (main) el nombre de un archivo binario y una lista de obras sociales. Luego se deberá mostrar continuamente al usuario un menú con las siguientes tres opciones:

- Agregar un nuevo paciente a la cola ó arreglo dinámico de espera desde la entrada estándar (stdin).
- Imprimir todos los pacientes por pantalla (stdout).
- Generar un archivo de texto (con nombre igual al pasado por línea de comandos) con los pacientes filtrados por obra social.

Utilice las funciones desarrolladas en los punteros anteriores para el desarrollo de cada opción del menú.

Ejemplo:



Apellido y Nombres	Legajo	# de Hojas	Profesor

```
$ ./programa mi_archivo.bin osplad osecac oscomm
Menú principal:
1. Agregar un nuevo paciente (desde stdin).
2. Imprimir todos los pacientes y su información por la salida estándar (stdout).
2. Generar un archivo de texto con los pacientes filtrados por obra social.
Elija una opción (1, 2 o 3):
```

Notas de implementación:

- Elegir e implementar el ejercicio 1a ó el ejercicio 1b. **Implementar ambos no sumará mayor puntaje a la evaluación.**
- Indique la forma de compilar y linkear todos los archivos fuente (.c) desarrollados.
- Coloque lo pertinente en el archivo encabezado (.h) correspondiente.
- Comente adecuadamente el código.
- No utilice variables globales.
- Ninguna función puede superar las 40 líneas de código.
- Verifique todas las condiciones de error.
- Utilice constantes simbólicas (#define) para las constantes.
- Haga un uso adecuado de la memoria.

Condición de aprobación:

- Sección Teórica: 2 ejercicios correctamente resueltos.
- Sección Práctica: 2 ejercicios correctamente resueltos + 1 ejercicio correctamente planteado **sin errores conceptuales**.

Duración del examen: 1:45 horas.